

PLANO DE DESENVOLVIMENTO

Yu-Gi-Oh! na Unity a partir do zero

Integração com ygopro-core, CardScripts e BabelCDB para um catálogo inicial de cerca de 200 cartas

Objetivo do documento

Fornecer ao Codex uma ordem precisa para criar um projeto novo em Unity 6000.5.0f1, sem desenvolver um Core próprio de regras e sem criar 200 efeitos manualmente em C#. O ygopro-core será a autoridade do duelo; os scripts Lua definirão os efeitos; a Unity cuidará da interface, campo, modelos 3D, animações, áudio e escolhas do jogador.

Versão do plano: 1.0 - 28 de julho de 2026

1. Decisão fundamental antes de iniciar

Não serão criadas 200 classes C# de efeitos

Para cartas oficiais já existentes no Project Ignis, “adicionar uma carta” significa registrar o código oficial, disponibilizar o script c<ID>.lua, carregar seus dados e associar a arte visual. Somente cartas fanmade ou cartas sem script exigem criação de um novo script Lua.

O projeto deve nascer com uma única fonte de verdade para as regras:

```

Unity / C#
- UI, cenas, campo, modelos 3D, animações, áudio e input
    |
    v
Ponte C# <-> API nativa
    |
    v
ygopro-core / C++
- estado, fases, correntes, legalidade, custos, alvos e resolução
    |
    +--> CardScripts / Lua: efeitos das cartas
    +--> BabelCDB / SQLite: dados e textos das cartas

```

O plano considera inicialmente Windows x64 no Editor e em build Standalone. Android, iOS e servidor serão etapas posteriores.

2. Fontes oficiais e arquivos necessários

Componente	Endereço
Motor de regras	https://github.com/edo9300/ygopro-core
Scripts das cartas	https://github.com/ProjectIgnis/CardScripts
Banco de dados	https://github.com/ProjectIgnis/BabelCDB
Cliente de referência	https://github.com/edo9300/edopro
API do Core	https://github.com/edo9300/ygopro-core/blob/master/ocgapi.h
Tipos da API	https://github.com/edo9300/ygopro-core/blob/master/ocgapi_types.h
Constantes e mensagens	https://github.com/edo9300/ygopro-core/blob/master/ocgapi_constants.h
Unity - DllImport	https://docs.unity3d.com/6000.5/Documentation/Manual/plugin-native-dllimport.html
Unity - Plugin Inspector	https://docs.unity3d.com/6000.5/Documentation/Manual/plugin-inspector.html
Unity - StreamingAssets	https://docs.unity3d.com/6000.5/Documentation/Manual/StreamingAssets.html
Licença AGPL	https://www.gnu.org/licenses/agpl-3.0.html

O Rulebook_v9_en.pdf pode ser mantido em Documentation/Rules como referência básica. Interações modernas e específicas devem ser validadas pelo texto oficial da carta, scripts atuais e comportamento do Core.

3. Mensagem de comando para o Codex

Envie este documento ao Codex junto com o texto abaixo:

Leia integralmente o PDF anexado. O projeto será criado do zero na Unity 6000.5.0f1. Use o ygotpro-core como única autoridade das regras. Não crie um Core alternativo em C#. Execute as fases na ordem indicada e pare ao final de cada marco para apresentar:

- arquivos criados e alterados;
- comandos executados;
- testes realizados;
- resultados e erros;
- próximo passo recomendado.

Comece somente pelas Fases 0, 1 e 2. Não construa a interface final e não importe as 200 cartas antes de o duelo headless básico estar funcionando.

4. Fase 0 - Definir o catálogo de 200 cartas

1. Criar uma planilha ou CSV chamado Documentation/CardCatalog.csv.
2. Listar exatamente as aproximadamente 200 cartas que farão parte da primeira versão.
3. Para cada carta, registrar: nome, código oficial, tipo, deck de origem, script encontrado, registro no banco encontrado, imagem disponível e status de teste.
4. Separar cartas oficiais de cartas fanmade. Não misturar os dois fluxos.
5. Classificar as cartas por complexidade: simples, intermediária, contínua, gatilho, rápida, negação, substituição, Extra Deck e excepcional.
6. Escolher 12 cartas representativas para o primeiro corte vertical.
7. Não importar o catálogo inteiro até que as 12 cartas funcionem de ponta a ponta.

Campo CSV	Exemplo
official_code	5318639
name	Mystical Space Typhoon
source	official
script_path	official/c5318639.lua
database_found	true
art_asset	Assets/Game/Cards/Art/5318639.png
test_status	pending

5. Fase 1 - Criar e versionar o projeto Unity

1. Criar um projeto Unity 6000.5.0f1 usando um template apropriado ao visual desejado, preferencialmente URP para um campo 3D moderno.
2. Ativar Git imediatamente e usar um .gitignore próprio para Unity.
3. Fazer o primeiro commit antes de integrar dependências.
4. Criar uma branch feature/ocgcore-bootstrap.
5. Configurar uma cena Bootstrap, uma cena Duel e uma cena CardLab para testes isolados.
6. Criar assembly definitions desde o começo para separar Runtime, Editor e Tests.
7. Adicionar o Unity Test Framework e confirmar um teste EditMode e um PlayMode simples.

```
ProjectRoot/
  Assets/
    Game/
      DuelEngine/
      Plugins/
      StreamingAssets/
      Tests/
```

```
Packages/  
ProjectSettings/  
ThirdParty/  
Tools/  
Documentation/  
LICENSES/
```

6. Fase 2 - Adicionar as dependências como submódulos

```
git submodule add https://github.com/edo9300/ygopro-core.git ThirdParty/ygopro-core  
git submodule add https://github.com/ProjectIgnis/CardScripts.git ThirdParty/CardScripts  
git submodule add https://github.com/ProjectIgnis/BabelCDB.git ThirdParty/BabelCDB  
git submodule update --init --recursive
```

1. Manter ThirdParty fora de Assets para a Unity não importar os fontes dos repositórios.
2. Registrar os hashes exatos em ThirdPartyVersions.json.
3. Copiar as licenças para LICENSES sem modificar o texto.
4. Criar THIRD_PARTY_NOTICES.md, MODIFICATIONS.md e SOURCE_CODE.md.
5. Tratar Core, scripts e banco como um pacote versionado; não atualizar um componente isoladamente.

7. Fase 3 - Compilar o ygopro-core para Windows x64

Pré-requisitos: Visual Studio 2022, Desktop development with C++, Windows SDK e Git.

1. Atualizar os submódulos internos do ygopro-core.
2. Usar os scripts de geração fornecidos pelo commit fixado no repositório; confirmar o método atual no README antes de executar.
3. Gerar a solução para Visual Studio e compilar a biblioteca compartilhada em Release x64.
4. Localizar a DLL e suas dependências reais, sem supor nomes ou caminhos.
5. Copiar o binário para Assets/Plugins/Windows/x86_64/.
6. No Plugin Inspector, habilitar Editor e Standalone Windows x86_64 e desabilitar outras plataformas.
7. Criar um teste C# que chame OCG_GetVersion por DllImport.
8. Não prosseguir enquanto o Editor não carregar e retornar a versão sem AccessViolationException.

Marco 1

A Unity consegue carregar a biblioteca e imprimir a versão do OCG API. Nenhum duelo ainda é criado.

8. Fase 4 - Criar a ponte C# segura

```
Assets/DuelEngine/Runtime/
  Abstractions/
  Interop/
  Core/
  Protocol/
  Data/
  Scripts/
  State/
  Presentation/
  Diagnostics/
```

A camada Interop deve ser a única parte que conhece ponteiros, Marshal e DllImport.

- **OcgCoreNative.cs:** declara a API C do commit utilizado.
- **OcgDuelHandle.cs:** garante destruição segura da sessão nativa.
- **OcgDuelEngine.cs:** expõe uma API C# de alto nível.
- **OcgCallbacks.cs:** mantém referências fortes para leitores de dados, scripts e logs.
- **InteropLayoutTests.cs:** confere tamanho e offsets das estruturas entre C# e C++.

```
public interface IDuelRulesEngine : IDisposable
{
    void Create(DuelConfiguration configuration);
    void AddCard(DuelCardPlacement card);
    void Start();
    DuelProcessResult Process();
    void SubmitResponse(ReadOnlySpan<byte> response);
    DuelSnapshot QuerySnapshot();
}
```

Não coloque DllImport em MonoBehaviours. Nenhum script de UI deve receber IntPtr.

9. Fase 5 - Preparar dados e scripts

9.1 Banco das cartas

1. Inspecionar o schema real do arquivo .cdb com PRAGMA table_info antes de criar consultas.
2. Criar uma ferramenta externa em Tools/CardDbCompiler para converter somente os dados necessários das 200 cartas para cards.bin e card-texts.json.
3. Preservar código, tipo, nível/rank, atributo, raça/tipo, ATK, DEF, escalas, marcadores Link, nome, texto e strings auxiliares.
4. Validar códigos duplicados, registros ausentes e textos ausentes.

5. Implementar o callback OCG_DataReader com memória não gerenciada segura, incluindo setcodes quando necessário.
6. Implementar OCG_DataReaderDone para liberar toda alocação sem double free.

9.2 Scripts Lua

1. Criar Tools/SyncYgoContent.ps1.
2. Copiar utilitários globais e apenas os scripts das 200 cartas para Assets/StreamingAssets/Ygo/Scripts/.
3. Manter scripts fanmade em Assets/StreamingAssets/Ygo/CustomScripts/.
4. Buscar primeiro em CustomScripts e depois em Scripts/official.
5. Não traduzir os scripts Lua para C#.
6. Validar que todo código do catálogo possui script ou está explicitamente marcado como carta sem efeito.

10. Fase 6 - Fazer um duelo headless antes da interface

Crie um runner que funcione sem campo 3D, sprites, animações ou botões finais.

1. Criar as opções do duelo e uma seed determinística.
2. Registrar callbacks de dados, scripts e logs.
3. Criar a sessão nativa.
4. Adicionar dois Main Decks e os Extra Decks.
5. Iniciar o duelo.
6. Executar OCG_DuelProcess.
7. Copiar imediatamente o buffer retornado por OCG_DuelGetMessage.
8. Decodificar inicialmente apenas mensagens de começo, compra, turno, fase e fim.
9. Enviar respostas fixas de teste quando o Core solicitar escolhas simples.
10. Finalizar e destruir a sessão sem vazamentos.

Marco 2

Dois decks de teste iniciam uma partida headless, compram a mão e avançam pelo menos um turno usando uma seed repetível.

11. Fase 7 - Decodificar mensagens e codificar respostas

A integração real depende de converter o protocolo binário em objetos C# tipados.

Grupo	Mensagens prioritárias
Fluxo	START, NEW_TURN, NEW_PHASE, WIN
Cartas	DRAW, MOVE, SHUFFLE, POS_CHANGE
Invocações	SUMMONING, SUMMONED, SPSUMMONING, SPSUMMONED
Correntes	CHAINING, CHAIN_SOLVING, CHAIN_SOLVED, CHAIN_END
Escolhas	SELECT_CARD, SELECT_OPTION, SELECT_YESNO, SELECT_POSITION, SELECT_PLACE
Batalha	ATTACK, BATTLE, DAMAGE, RECOVER

1. Usar common.h, ocgapi_constants.h, EDOPro e um codec atual como referências do mesmo commit.
2. Criar BinaryMessageReader com checagem de limites.
3. Criar uma classe C# para cada mensagem suportada.
4. Criar DuelResponseEncoder para cada solicitação de escolha.
5. Capturar buffers reais em testes e armazená-los como fixtures.
6. Falhar explicitamente ao receber mensagem desconhecida; nunca ignorá-la.
7. Adicionar logs com código numérico, posição do buffer e contexto.

12. Fase 8 - Projetar o estado para a Unity

```
ygopro-core
-> DuelMessageDecoder
-> DuelStateProjector
-> DuelPresentationQueue
-> Campo, cartas, UI e animações
```

O DuelStateProjector cria uma cópia de apresentação, mas não substitui o estado do Core.

- Consultar o Core após sequências críticas para verificar divergências.
- Identificar cartas por referência lógica, não por GameObject.
- Enfileirar animações somente depois da mensagem do motor.
- Nunca atrasar a resolução da regra para esperar uma animação.
- Permitir reconstruir toda a cena visual a partir de um snapshot do Core.

13. Fase 9 - Construir o corte vertical com 12 cartas

O primeiro grupo deve cobrir várias famílias de comportamento:

Quantidade	Categoria
2	Monstros Normais
2	Efeitos de Ignição
2	Efeitos de Gatilho
1	Efeito Rápido
1	Magia Normal
1	Magia Rápida
1	Armadilha
1	Monstro de Fusão/Sincro/Xyz/Link
1	Efeito contínuo ou substituição

1. Criar dois decks pequenos que garantam o acesso às 12 cartas.
2. Testar compra, Invocação, ativação, custo, alvo, resposta, Corrente e resolução.
3. Criar uma UI provisória de escolhas e um campo funcional mínimo.
4. Usar artes temporárias quando necessário; a lógica vem primeiro.
5. Não iniciar a importação das 200 cartas antes de todas as 12 concluírem um duelo completo.

Marco 3

Um duelo completo pode ser jogado do início à vitória usando as 12 cartas representativas, com escolhas reais do jogador e Correntes funcionais.

14. Fase 10 - Criar o importador das 200 cartas

Depois do corte vertical, automatize o conteúdo em vez de cadastrar manualmente cada carta na Unity.

1. Ler Documentation/CardCatalog.csv.
2. Validar código no banco e script no diretório.
3. Gerar CardVisualData ScriptableObjects ou um catálogo binário somente para apresentação.
4. Associar arte, moldura, nome, texto e código oficial.
5. Gerar relatórios de arquivos ausentes e duplicados.
6. Não armazenar lógica de efeito no ScriptableObject.
7. Importar em lotes de 20 a 25 cartas.
8. Executar testes automáticos após cada lote.
9. Não modificar scripts oficiais para corrigir problemas visuais ou de interface.

```
CardVisualData
  officialCode
  artwork
  frameStyle
  summonVfx
  activationSfx
  optionalPresentationTags

// Nenhum método ResolveEffect() ou regra de duelo.
```

15. Fase 11 - Estratégia de testes para as 200 cartas

Nem todas as 200 cartas exigem o mesmo número de testes, mas todas precisam de validação mínima.

Camada	Cobertura
Interop	DLL, layouts, callbacks, memória e lifecycle
Dados	registro, texto, script e setcodes
Protocolo	decoder/encoder e buffers malformados
Core	duelo, turnos, fases, Correntes e snapshots
Cartas simples	carrega, pode ser usada e produz o resultado básico
Cartas complexas	casos legais, ilegais, alvo perdido, negação e limite de uso
Integração visual	mensagem -> estado projetado -> animação
Regressão	todo lote anterior continua funcionando

Classifique os testes por risco:

- **Nível A - simples:** monstros normais e efeitos diretos; smoke test automatizado.
- **Nível B - intermediário:** busca, compra, Invocação e modificações temporárias; testes positivos e negativos.
- **Nível C - crítico:** negação, substituição, efeitos contínuos, múltiplos gatilhos, Extra Deck e interações; cenários completos de Corrente.

16. Fase 12 - Deck builder e validação

1. Construir o deck builder somente depois de o catálogo e o duelo estarem estáveis.
2. Usar o código oficial como identificador persistente.
3. Validar Main Deck e Extra Deck conforme o formato adotado pelo projeto.
4. Exibir somente as 200 cartas habilitadas no catálogo inicial.
5. Salvar decks em um formato textual e versionado, fácil de depurar.
6. Antes do duelo, validar que todos os códigos existem no banco e possuem recursos necessários.

17. Fase 13 - Interface, arena e modelos 3D

Como o projeto começa do zero, a apresentação deve ser construída sobre eventos do motor, não o contrário.

- CardViewRegistry cria ou localiza a representação visual de cada referência lógica.
- DuelAnimationQueue mantém a ordem visual sem controlar regras.
- PlayerChoicePresenter destaca somente candidatos fornecidos pelo Core.
- A arena contém pontos de encaixe por zona, mas não decide legalidade.
- O campo pode ser reconstruído integralmente após falha visual ou reconexão.
- Animações excepcionais são associadas por código oficial em dados visuais.

18. Fase 14 - Build e distribuição amistosa

1. Gerar primeiro um build Windows x64 Development para os testes.
2. Confirmar que DLL, scripts e dados são incluídos nos caminhos esperados.
3. Testar em uma segunda máquina sem Visual Studio instalado.
4. Incluir licenças, avisos e instruções de acesso ao código-fonte correspondente.
5. Registrar versão da Unity, commit do projeto, commit do Core, scripts e banco em uma tela de diagnóstico.
6. Criar um pacote de logs para que os colegas possam enviar falhas reproduzíveis.
7. Somente depois gerar um build não Development para o torneio amistoso.

19. Cartas oficiais versus cartas fanmade

Situação	Ação
Carta oficial com script	Importar código, dados, script e arte; testar.
Carta oficial sem script no pacote fixado	Verificar atualização compatível ou criar script próprio em CustomScripts.
Carta fanmade simples	Criar entrada de dados e script Lua usando a API do ocgcore.
Carta fanmade complexa	Especificar timing/custo/alvo/resolução, criar testes e só então escrever Lua.
Carta sem efeito	Dados e arte bastam; confirmar que o tipo está correto.

Regra de manutenção

Nunca edite diretamente o script oficial dentro de ThirdParty/CardScripts. Sobrescritas e fanmade devem ficar em CustomScripts, com documentação e testes próprios.

20. Ordem final resumida para o Codex

1. Definir o catálogo das 200 cartas.
2. Criar o projeto Unity e o Git.
3. Adicionar dependências e licenças.
4. Compilar e carregar a DLL.
5. Criar a ponte C# segura.
6. Preparar banco e scripts.
7. Executar um duelo headless.
8. Decodificar mensagens e respostas.
9. Projetar estado para a Unity.
10. Concluir o corte vertical de 12 cartas.
11. Criar o importador automático.
12. Adicionar as 200 cartas em lotes.
13. Criar deck builder e interface completa.

- 14. Executar testes e regressões.
- 15. Gerar o build para outra máquina.
- 16. Somente depois planejar multiplayer e mobile.

21. Critérios de aceitação

- A Unity carrega a biblioteca nativa sem erro.
- O motor inicia e termina um duelo completo.
- Correntes, custos, alvos e respostas são decididos pelo ygopro-core.
- As 200 cartas estão no catálogo e têm código oficial válido.
- Toda carta oficial habilitada possui banco e script, salvo exceções documentadas.
- Não existem 200 classes C# de efeitos.
- A interface pode ser reconstruída a partir do estado do motor.
- Nenhuma animação determina o resultado de uma regra.
- O build funciona em outra máquina.
- Licenças e código-fonte correspondente estão acessíveis.
- Atualizações futuras são feitas por pacote versionado e com testes de regressão.

22. O que não fazer

- Não desenvolver primeiro um Core próprio em C# para depois substituí-lo.
- Não traduzir todos os scripts Lua para C#.
- Não começar pelas 200 cartas antes do corte vertical.
- Não criar a interface final antes do duelo headless.
- Não usar GameObjects como estado oficial.
- Não importar todo CardScripts para Assets sem filtro.
- Não atualizar apenas o banco, apenas os scripts ou apenas o Core.
- Não ignorar mensagens desconhecidas e erros Lua.
- Não implementar Android simultaneamente ao protótipo Windows.
- Não afirmar que uma carta está correta sem teste reproduzível.

Resultado esperado

Ao final, o projeto terá cerca de 200 cartas disponíveis, mas os efeitos não estarão codificados manualmente em 200 arquivos C#. A Unity será um cliente visual em C#, enquanto o ygopro-core e os CardScripts executarão as regras e os efeitos. Isso permite ampliar o catálogo posteriormente sem reconstruir o motor.